



ENTROPY

A relational engine, crash-tested by *deterministic simulation*

Slotted-page storage, a latch-crabbing B+ tree, MVCC snapshot isolation, write-ahead logging, and ARIES-style recovery, written from scratch in C++20 and driven through seeded, replayable fault injection with invariant checks after every crash.

ABSTRACT

Entropy is a relational database engine written from scratch in C++20: slotted-page storage under a latch-crabbing B+ tree, MVCC snapshot isolation, write-ahead logging, and ARIES-style three-pass recovery, fronted by a SQL parser, a cost-based planner, and volcano executors.

This document examines the claim databases find hardest to earn: that a crash at any instant loses nothing committed and leaks nothing uncommitted. Entropy tests it the way FoundationDB does, with a deterministic simulator that runs the production stack over fault-injecting storage devices. One 64-bit seed derives independent PRNG streams for the workload, the page device, and the log store; a crash resolves every unsynced write to lost, torn, or kept; recovery runs on the damaged image; and an oracle checks every committed row, byte for byte, against what must have survived. A failing seed replays identically until it is understood.

The method paid for itself immediately. Seed sweeps surfaced two crash-path bugs that more than 550 passing unit and integration tests had never touched: cleanly aborted transactions resurrected by repeat-history redo, and torn page writes that recovery signed off as success while committed bytes read back as zeros. Both are analyzed here to root cause, both are fixed (compensation log records on the abort path; CRC-32 page checksums with a recovery-mode rebuild), and both fixes are locked by the schedules that found them, re-verified across 40 seeds on every push. The engine holds 0.90x SQLite on 1k-row transactional insert batches on the committed benchmark run, and chapter 07 states plainly what does not work yet.

22,500

non-empty lines of C++20, all first-party

9 × 40

schedules × seeds swept on every CI push

95 / 200

seeds failing before the torn-page fix; 0 after

0.90x

vs SQLite, 1k-row transactional insert batch (M2)

CONTENTS

01	Crash recovery as a falsifiable claim	04
02	Two bugs the test suite never caught	07
03	The engine under test	12
04	From SQL text to a costed plan	16
05	An instrument panel for crashes	18
06	Measured, on two machines	20
07	What this engine does not do	22
A	Reproduction and colophon	23

All figures are drawn from the repository: source, issues, pull requests, commit history, committed benchmark runs, and live output of the referenced builds.

PART I

Faults, by design

01 Crash recovery as a falsifiable claim

02 Two bugs the test suite never caught

Crash recovery as a falsifiable claim

Recovery carries the engine's hardest promise: after a crash at any instant, every committed row is still there, byte for byte, and no uncommitted write shows. Entropy tests that promise the way FoundationDB pioneered it: the real engine runs over simulated storage devices, every fault is drawn from a seeded generator, and a failing run replays exactly from one 64-bit integer.

1.1 Why determinism

A recovery bug lives in a window a few writes wide: the crash must land after the WAL buffer fills but before the fsync, or while half of a 4 KB page is on disk. Killing processes in a loop hits such windows by luck and cannot hit them again on demand, so the failure arrives without a reproduction and leaves without a regression test. The simulator inverts that economics. It decides when the crash happens and what the disk keeps, so the search over crash timings is systematic, and every failure is a permanent artifact: a seed.

Nothing in the fault model reads a wall clock or an unseeded source. Two runs of the same seed under the same schedule produce equal fault logs, equal redo and undo counts, and an identical output line: the determinism a debugger needs and a CI gate can enforce.

FAULT.HPP

`derive_seed(seed, salt)` applies the SplitMix64 finalizer: multiply-xor-shift mixing, stateless and deterministic. Independent streams from one seed, no draw-order interference.

SCALE

The schedule sweep in CI runs nine schedules across seeds 1 to 40: 360 crash-recover-verify cycles on every push.

1.2 One seed, three streams

`run_schedule` derives three independent PRNG streams from the run's seed with SplitMix64 mixing: salt `0x1` for the workload, `0x2` for the page device, `0x3` for the log store. Each derived value seeds its own `std::mt19937_64`, so workload decisions never perturb fault decisions and vice versa.

Fault probabilities are integer parts-per-thousand compared against `rng() % 1000`, never floating point, so every threshold is exact and portable across platforms. A disabled fault never consumes a draw: switching one knob off cannot shift the sequence any other consumer sees.



FIG 1.1 Seed derivation. One CLI-supplied integer fans out into three isolated Mersenne Twister streams; each simulated device and the workload own exactly one.

1.3 The storage seam

The simulator swaps only the lowest layer. `SimDiskManager` and `SimLogStore` stand in for the file-backed page device and log store; everything above them (buffer pool, WAL manager, lock manager, MVCC version store, transaction manager, table heap) is the production code. The assembled stack keeps the engine's WAL-before-page steal hook, so the simulation exercises the same durability ordering the real engine enforces: no data page reaches disk before the log that describes it.

A crash freezes the device and resolves the fate of every write that was issued but never fsynced. One roll per write (`draw_fate`) decides among three outcomes: the write is **lost**, the write is **torn**, or it happened to reach durable media **intact**. A torn page keeps a prefix or a suffix of the new bytes across a boundary drawn from $[1, 4095]$, the other side reverting to the pre-crash image. A torn WAL tail keeps `rng() % (len+1)` bytes of the unsynced tail, and the cut lands mid-record, so the log reader's partial-record truncation is exercised end to end. Writes the engine fsynced are never touched: the model attacks exactly what a real power loss attacks.

1.4 Recovery under an oracle

While the workload runs, an oracle records the outcome each transaction reached: committed rows with their exact payloads, aborted and in-flight writes that must vanish. After the crash, the damaged images reopen on fresh, fault-free devices and `RecoveryManager::recover()` runs: the same analysis, redo, and undo the production engine runs at startup. Then `check_invariants` compares the recovered heap against the oracle:

`committed_present` requires every committed row to exist with a byte-identical payload. `no_uncommitted_visible` requires that no in-flight or rolled-back write is visible. A non-ok status from recovery itself is recorded as `recovery_error`. Each run emits one line of JSONL with a stable schema; wall-clock recovery time is opt-in (`--timing`) precisely because it is the one non-reproducible field.

TOOLS/ENTROPY-SIM

The CLI sweeps seeds and prints one line per run. A sweep is machine-checkable output, not a green light: the JSONL stream is the artifact.

ONE RUN, ONE LINE

```
{ "seed": 7, "schedule": "mixed",
  "faults_injected": 13,
  "crash_point":
    "mixed_steal_lost_inflight",
  "outcome": "pass",
  "invariants_failed": [],
  "ops": 57, "redo_ops": 40,
  "undo_ops": 0 }
```



FIG 1.2 One run under `run_schedule`. The live stack runs on injecting devices; `crash()` applies the scheduled damage to unsynced state; recovery runs on clean devices and an oracle passes judgment.

1.5 Named schedules

A schedule fixes everything except the seed: transaction count, whether a loser transaction is left in flight, whether dirty pages are stolen to disk before the crash, and the fault probabilities. Nine presets place the crash at the seams where recovery earns its keep: between the WAL flush and the page flush, mid-WAL-tail, after a steal of committed pages, and under transient write errors.

SCHEDULE	CRASH POINT	INJECTED FAULTS	CONTRACT
torn_wal_tail	in-flight txn, WAL tail at risk	tail lost 50% · torn 50%	both must fire
lost_page_write_after_commit	committed pages stolen, unsynced	every page write lost	loss must fire
crash_between_wal_and_page_flush	WAL durable, pages in flight	every page write lost	loss must fire
undo_durable_loser	loser's records durable, no COMMIT	page writes lost	undo_ops > 0, every seed
durable_survives_intact	clean fsync'd baseline	none (control)	zero faults, every seed
transient_write_errors	flush under failing writes	30% IO errors + losses	both must fire
mixed	steal + lost/torn WAL tail	losses + tail lost 70% / torn 30%	three kinds must fire
live_abort_repro	aborts, then steal + loss	page writes lost	≥1 abort, every seed
torn_page_write	committed pages torn, unsynced	every page write torn	tear must fire

1.6 Contracts against vacuity

A fault-injection suite has a quiet failure mode of its own: the advertised fault stops firing (a refactor moves an fsync, a buffer stops overflowing) and the suite stays green while testing nothing. Every schedule therefore carries a contract the sweep asserts alongside the invariants. `must_fire` lists fault kinds that must occur at least once across the sweep's seeds; `expect_undo` requires recovery's undo phase to do real work on every seed; `expect_aborts` requires at least one live abort per seed; `expect_zero_faults` pins the clean-baseline control at exactly zero injected faults.

The checker itself is under test. A tenth schedule, `skip_recovery`, repeats `lost_page_write_after_commit` but skips recovery. Committed rows then exist only in the WAL, and the sweep requires the invariant checker to report the loss; a checker that cannot detect a real loss would prove nothing by staying green.

SIZING

A minimal insert's WAL record is ~79 bytes, so ~830 inserts overflow the 64 KiB WAL buffer. Schedules that need a genuinely unsynced tail run the in-flight loser for 1,200 inserts, forcing a mid-transaction overflow flush onto the device.

HIGHLIGHTED ROWS

The two amber schedules are regression locks for real bugs the simulator caught. Chapter 02 tells both stories.

SWEEP SUMMARIES · THIS BUILD, 200 SEEDS PER SCHEDULE, VERBATIM

```
entropy-sim: schedule=torn_wal_tail seeds=200 start=1 200 passed, 0 failed, 0 errored
faults fired: lost_wal_tail=94 torn_wal_tail=106 undo_exercised_runs=106

entropy-sim: schedule=undo_durable_loser seeds=200 start=1 200 passed, 0 failed, 0 errored
faults fired: lost_page_write=2200 undo_exercised_runs=200

entropy-sim: schedule=mixed seeds=200 start=1 200 passed, 0 failed, 0 errored
faults fired: lost_page_write=200 lost_wal_tail=139 torn_wal_tail=61 undo_exercised_runs=61

entropy-sim: schedule=durable_survives_intact seeds=200 start=1 200 passed, 0 failed, 0 errored
faults fired: none undo_exercised_runs=0
```

Two bugs the test suite never caught

The engine came first, by about seven months. Storage, the B+ tree, MVCC, the WAL, and recovery each carried unit and integration suites, more than 550 tests passing, when the simulator ran its first wide seed sweeps. Those sweeps found two bugs in the crash path of that tested code. Both violated atomicity or durability. Both were invisible to example-based tests, because both lived in states no hand-written test thought to construct.

Each case followed the same arc. A schedule failed on specific seeds. The seed replayed the failure deterministically until the root cause fell out. A fix landed. The schedule that found the bug joined the permanent sweep, where it re-runs the scenario across 40 seeds on every CI push. The failure, the fix, and the guarantee are all in the record: issues #75, #81, and #86, and the fix commits eebf21c and 8bd6db2.

2.1 Case I • the resurrected abort

The `live_abort_repro` schedule lets transactions abort during normal operation (400% per transaction), commits the rest, steals the committed pages to disk unsynced, and crashes so every one of those page writes is lost. Recovery must rebuild the table from the WAL alone. Under the pre-fix engine the sweep failed on **every seed: 40 of 40**, reporting `no_uncommitted_visible` (an aborted row came back) and, on seeds where a later transaction had reused the aborted row's slot, `committed_present` (the committed row was gone too).

The failure shrinks to three WAL records. A transaction begins, inserts a row, and aborts, all cleanly, no crash yet. The WAL now holds `BEGIN, INSERT, ABORT`. Recover that log onto a fresh buffer pool and the aborted row is present:

ISSUE #81 • MINIMAL REPRODUCTION

```
// WAL: BEGIN(1), INSERT(1, rid=(0,0), data), ABORT(1) – flushed, durable.
RecoveryManager rec(pool, wal, disk);
rec.recover();           // ok=1, redo_count=1, undo_count=0
// RID(0,0) is PRESENT after recovery – the aborted insert was resurrected.
```

Two failure signatures, one mechanism. Which one a seed reports depends only on whether a later committed transaction happened to reuse the slot the abort freed:

INVARIANT VIOLATED	OBSERVABLE DAMAGE	WHEN
<code>no_uncommitted_visible</code>	an aborted row is visible after recovery	every failing seed
<code>committed_present</code>	a committed row is missing outright	seeds with slot reuse

The failing schedule landed with the simulator itself, deliberately excluded from the passing sweep and annotated with the open issue numbers, so the failure survived as an executable artifact rather than as prose in a tracker. Promotion into the permanent sweep came later, inside the same pull request as the fix, which is what makes the schedule a regression lock rather than a demonstration.

WHY TESTS MISSED IT

The recovery unit tests hand-stamped page LSNs on their crafted pages, simulating maintenance the forward write path did not actually perform. The simulator assembled its pages the way production does, and the gap surfaced.

REPRODUCE

```
entropy-sim --schedule live_abort_repro
--seeds 40. Before the fix: 40/40 fail.
After: 40/40 pass, aborts firing on
every seed.
```

2.2 Root cause: repeat history, with nothing to reverse

ARIES-style recovery repeats history: the redo phase replays every logged data change, then the undo phase rolls back losers, transactions with no durable outcome record. A cleanly aborted transaction is not a loser. Its ABORT record is in the log, so analysis strikes it from the active-transaction table and undo never touches it. That is correct, but it rests on an assumption: the log must also carry the abort's undo work, so that repeating history repeats the compensation too.

Entropy's forward abort path broke that assumption. It undid the transaction's writes physically, in memory, flushed the compensated pages, and appended ABORT, logging no compensation records at all. As long as the compensated pages survived, nothing was wrong. The simulator's contribution was to take them away: steal the pages unsynced, lose them at the crash, and recovery rebuilds from a WAL that says INSERT and never says anything else. Page-LSN gating cannot save the rebuilt page either; a fresh frame carries LSN 0, so the gate never fires.

The slot-reuse variant turned an atomicity bug into data loss. After a live abort freed the row's slot, a later committed transaction reused it. During recovery, redo of the resurrected aborted insert occupied the slot first, and redo of the committed insert then found it taken and gave up: the aborted row present, the committed row gone.

2.3 Fix and guarantee

Commit eebf21c makes the abort path speak up in the log: one redoable compensation record (CLR) per undone write, stamped with the page's LSN and durable before the ABORT record appends. An undone insert logs a delete at the same RID, an undone delete logs an insert of the before-image, an undone update logs an update back to the before-image. Redo now replays each aborted mutation and its inverse, so the transaction leaves nothing behind even when its compensated pages never reached disk.

The schedule that found the bug became its regression lock. live_abort_repro joined the permanent sweep with expect_aborts asserted, so the coverage cannot silently decay: if aborts ever stop firing, the sweep fails on that alone. After the fix, 40 of 40 seeds pass in CI and a wider 500-seed sweep recorded 500 of 500.

ISSUE #75

Filed alongside: the forward write path never stamped page LSNs, so redo was never gated for forward-written pages. The CLR fix closes the abort hole; LSN stamping keeps redo idempotent when pages do survive.

ORDERING

The engine already made compensation durable before appending ABORT. The fix preserves that ordering for the CLR's, so a crash between the two leaves a loser, which undo already handles.

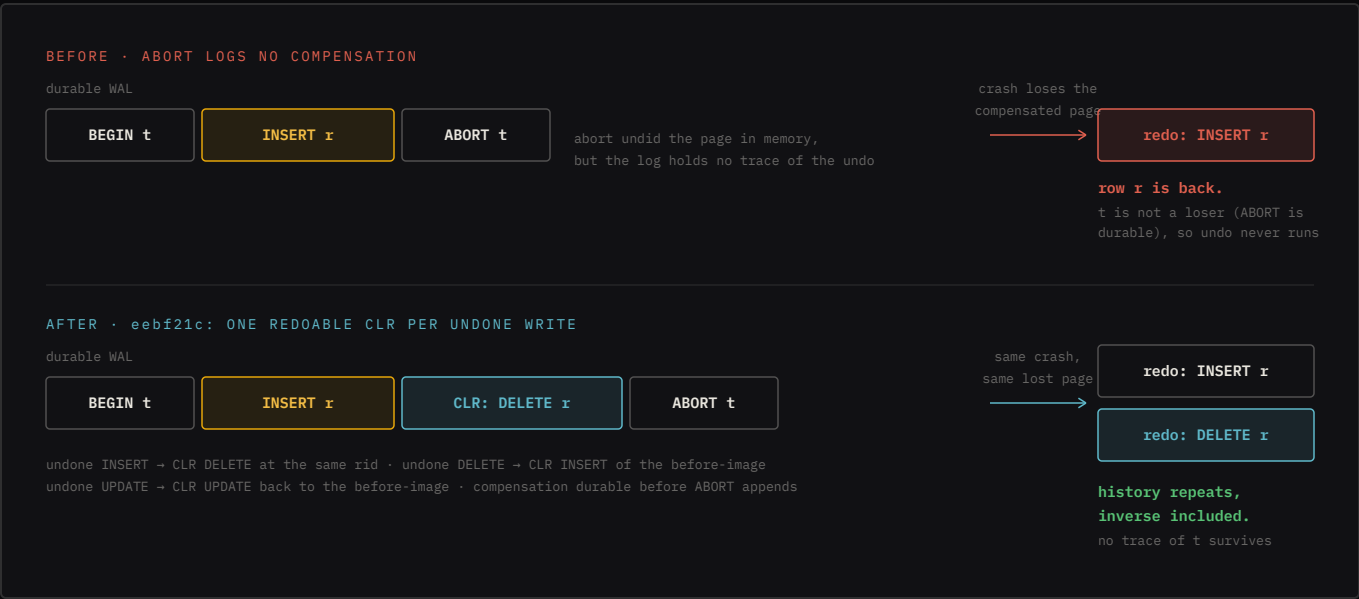


FIG 2.1 The resurrected abort. Before the fix, the durable log holds the aborted INSERT and no record of its undo; redo replays it and undo skips the transaction. After eebf21c, a compensation record rides in the log and repeat-history redo removes what it re-inserts.

2.4 Case II · the torn page that read back clean

A torn write is a crash persisting only part of a page write, a standard failure mode whenever the page size exceeds the device's atomic write unit. The `torn_page_write` schedule manufactures exactly that: commit fourteen transactions, steal the pages unsynced, and tear every unsynced page write at the crash, keeping a prefix or a suffix across a random boundary. Recovery then faces a disk full of half-old, half-new pages next to a fully durable WAL.

The pre-fix sweep failed `committed_present` on 95 of 200 seeds, and the split was not noise. Which side of the boundary survived decided the outcome. When the tear zeroed the header, redo saw an uninitialized page, re-created it, and replayed everything: those seeds passed. When the tear kept the first 128 bytes, the header and the slot directory, the page passed every check the engine had while its record region held zeros.

That geometry defeated recovery three ways at once. The surviving header still said `TABLE_PAGE`, so redo did not re-initialize the page. The slot directory still claimed the record was live, so redo's `insert_record_at` refused to write into an "occupied" slot. And nothing else validated the page, so `recover()` returned success with `redo=0`, `undo=0` while a committed row's bytes were zeros. Silent loss, signed off as a clean recovery.

THE RECORD

Issue #86, with the failing sweep attached. A wider pre-fix sweep recorded ~220 failing seeds of 500. Reproduce the fixed behavior: `entropy-sim --schedule torn_page_write --seeds 200`.

WHY TESTS MISSED IT

Every hand-written recovery test wrote whole pages. Only fault injection at the device seam produced a page that was half-persisted, and only the invariant checker noticed that a "successful" recovery had served zeros.

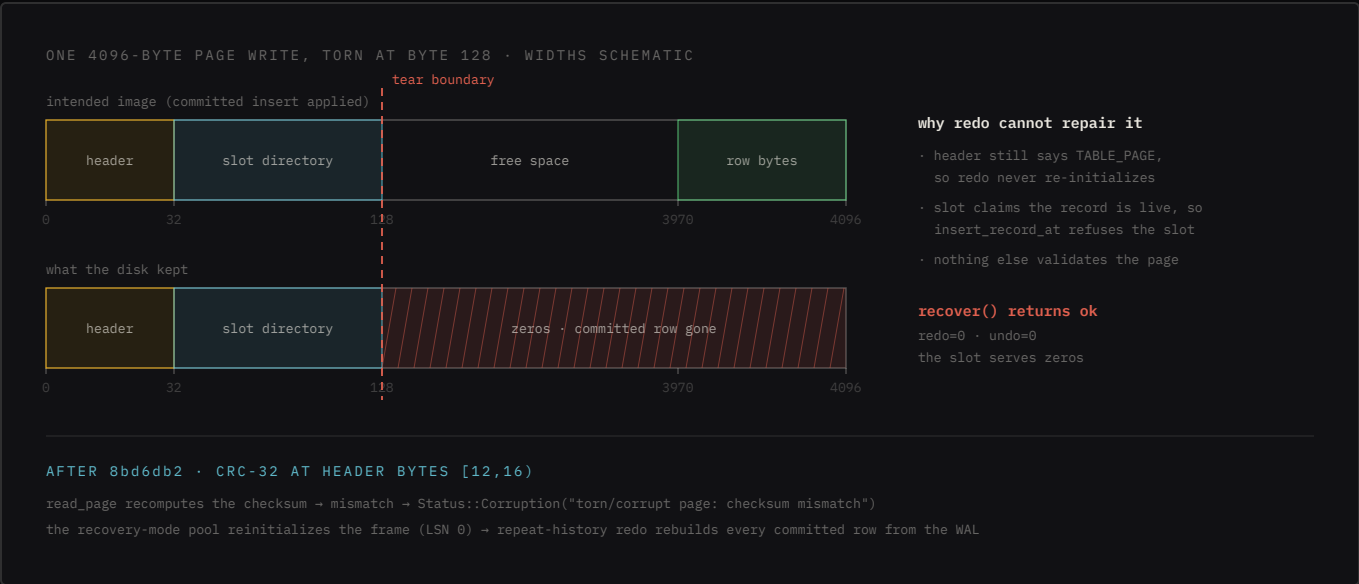


FIG 2.2 Anatomy of the fatal tear. The prefix that survives is precisely the part that convinces recovery the page is healthy: a valid header and a slot directory pointing at zeroed bytes.

2.5 Fix: make the page carry proof of itself

Commit 8bd6db2 gives every page an integrity stamp: a CRC-32 (IEEE polynomial 0xEDB88320) over the full 4096-byte image, stored in header bytes [12,16) and carved out of its own computation. The disk manager stamps on write and verifies on read. A torn page no longer matches its stamp, so the read returns `Status::Corruption` instead of serving half-persisted bytes as truth. An all-zero page verifies clean by rule, so freshly extended files need no stamp.

Detection alone would only turn silent loss into a loud error, so the buffer pool gained a recovery-mode seam. During recovery, a read that fails its checksum reinitializes the frame to a fresh, empty page with an invalid LSN, and repeat-history redo rebuilds every committed row onto it from the WAL. Outside recovery the same read stays a hard error. The fatal geometry from the sweep, header and slot directory intact over zeroed records, now takes the same path as a zeroed header: distrust the fragment, rebuild from the log.

The file-backed engine shipped the checksum default-off at first, on the belief that byte offset 12 collided with the B+ tree's `parent_page_id`. It does not: B+ tree nodes place their own header at offset 32, past the common page header, which puts `parent_page_id` at offset 44. Commit 62800ef corrected the analysis and enabled checksums by default, so the real engine now reports torn writes and bit rot as corruption on every read path, not only under simulation.

2.6 What the two cases share

Neither bug was exotic. One was a missing log record on a code path every database has; the other was a missing integrity check on a failure mode every storage engine faces. Both sat under a green suite of 550-plus tests, in the gap between the states tests construct and the states crashes construct. The schedules that exposed them now run as permanent locks, 80 crash-recover-verify cycles between them on every push, each with a contract that fails the sweep if its fault ever stops firing.

SCOPE

The CRC detects corruption; it does not repair it. Repair comes from the WAL, which is why the checksum ships with the recovery-mode rebuild rather than alone.

ALTERNATIVES

Full-page-write journaling (InnoDB's doublewrite, Postgres FPIs) repairs torn pages without a rebuild but pays for it on every checkpoint. For an engine whose WAL already replays whole histories, detect-and-redo is the cheaper contract.

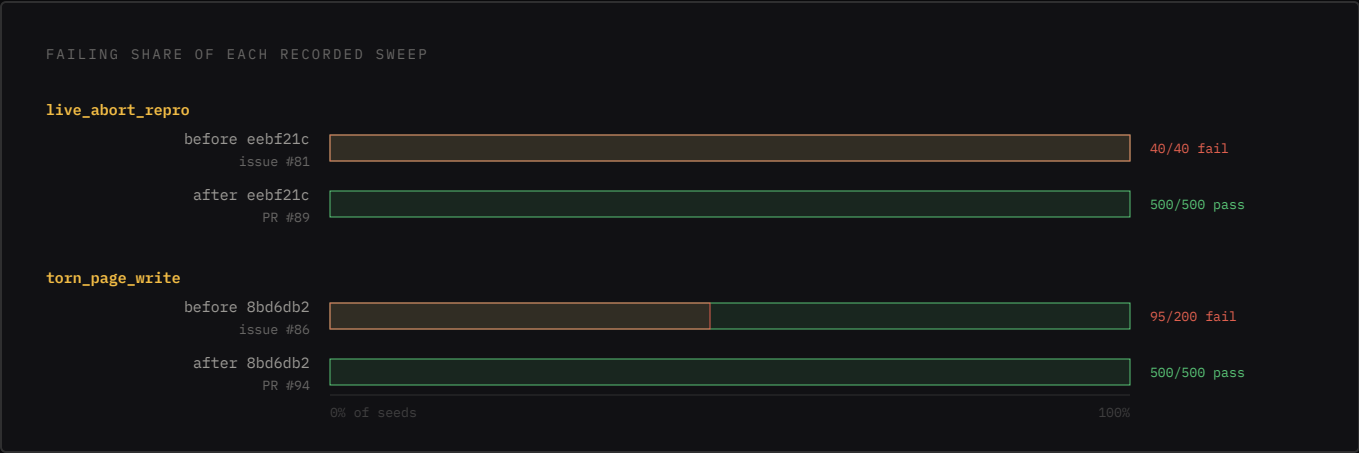


FIG 2.3 Recorded sweeps around each fix. `live_abort_repro` failed all 40 seeds before `eebf21c`; `torn_page_write` failed 95 of 200 before `8bd6db2`. Both now pass their full sweeps, re-verified on every CI run across seeds 1 to 40.

PART II

The machine

03 The engine under test

04 From SQL text to a costed plan

The engine under test

The simulator is only as interesting as the machine it attacks. Entropy is a relational engine written from scratch in C++20, about 22,500 non-empty lines of source under a 12,700-line test suite, with no third-party code on any hot path: the parser, the B+ tree, the buffer pool, the WAL, and recovery are all first-party. This chapter follows the write path from the page format up through the transaction stack, the same path the schedules in chapter 01 crash.

3.1 Slotted pages

Every durable structure lives on 4096-byte pages that begin with the same 32-byte header, laid out to be free of compiler padding and static-asserted at that size. A table page manages its interior as a classic slotted page: a slot array grows down from the header, four bytes per slot (offset and length), while record bytes grow up from the end. The two free-space fields in the header bound the contiguous gap between them.

Deleting a record clears its slot but leaves the bytes; a `compact()` pass reclaims the space when an insert would otherwise fail. Updates write in place when the new payload fits the old slot and relocate to fresh space when it does not. Slot indices are identity: a record's RID (page id, slot) survives in-place updates, and a slot freed by an uncommitted delete is held in reserve, not reused, until the deleter's fate is known.

The header's checksum field is the torn-write detector from chapter 02: a CRC-32 over the full page image with its own four bytes carved out, stamped on write, verified on every read, on by default in both the file-backed and simulated disk managers.

TUPLES

Rows serialize as a null bitmap, fixed-width columns in schema order, then length-prefixed varlen data. VARCHAR carries a 16-bit length; a page-sized cap keeps any tuple under 4 KB by configuration.

BUFFER POOL

1,024 frames by default (4 MiB). Pin counts guard reuse, an LRU list orders eviction, and a dirty victim is flushed through a hook that first forces the WAL to the page's LSN: steal is allowed, but no page may precede its log to disk.



FIG 3.1 The on-disk unit. Slots grow toward the records and records toward the slots; the header's checksum makes the whole 4 KB image self-verifying.

3.2 B+ tree, crabbed

The primary index is a B+ tree over 64-bit integer keys mapping to RIDs. Node capacity comes from the page arithmetic, not a tuning knob: after the shared header, a leaf holds up to 252 key-RID pairs and an internal node up to 337 separators. Leaves link both ways, so range scans ride sibling pointers without touching the tree above.

Concurrency is hand-over-hand latching on the pages themselves. Readers descend with shared latches, releasing each parent the moment the child is held, and hold at most two pages at any instant. Structural writers serialize on a single mutex, then crab down with exclusive latches, shedding every ancestor above the deepest node that cannot split or underflow. What a writer retains is exactly the path a restructure can touch, so a reader crossing a split either passes the parent before the change and finds the old child, or after it and finds the routed sibling. Never a half-made tree.

The deadlock argument is the part worth reading twice, and it lives as a 70-line comment in `b_plus_tree.cpp`. At the leaf level, every latch wait runs left to right; a writer that must turn left to borrow from a sibling first releases any right-hand latch it holds, because re-latching a leaf while holding one to its right can deadlock against a rightward chain scan. That exact interleaving surfaced as a live deadlock in review, and the release rule is its fix. Parent-pointer maintenance takes no latches at all: the field is written and read only under the writer mutex. And a grown root is published unlatched, through a release-store of the root page id that readers observe with an acquire load.

FRAME REUSE

The latch lives in the frame object, not the page bytes, and a latched page stays pinned, so the buffer pool can never evict and reuse a frame underneath a latch holder.

INTERNAL LEVELS

The writer's internal-level relatch order is not globally consistent, and does not need to be: the underflowing node's parent stays write-latched throughout, fencing every descending reader out of the subtree. A latch nobody else can request cannot join a cycle.

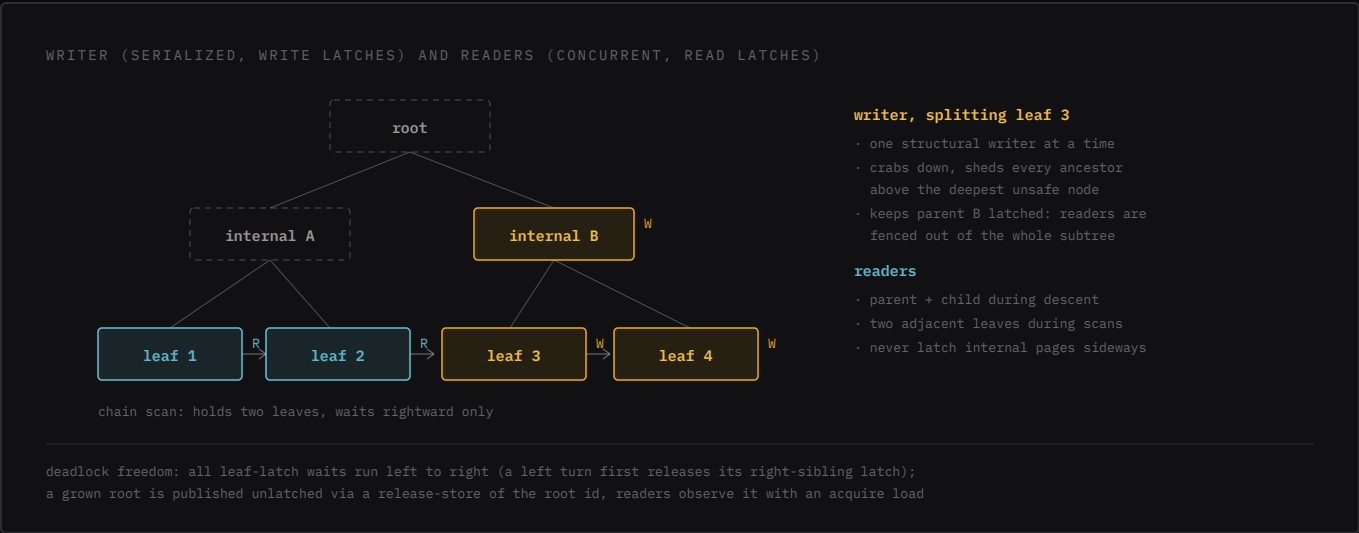


FIG 3.2 Latch crabbing. The writer holds its restructure path and nothing above; readers hold parent-child pairs on the way down and leaf pairs across scans, always waiting rightward.

3.3 MVCC snapshot isolation

Transactions read through snapshots. A single atomic counter issues timestamps; a transaction's `start_ts` fixes its view of the world, and each RID carries a chain of versions stamped with `begin_ts` and `end_ts`. A version is visible when it was created at or before the snapshot and not yet deleted from its view; a transaction's own uncommitted writes are always visible to itself. Readers never block writers and writers never block readers, because reading consults the chain, not a lock.

Write-write conflicts resolve first-updater-wins. A second writer touching the same RID aborts on any of four conditions: another transaction's uncommitted create, another's uncommitted delete, a committed delete of the current version, or a committed create newer than the writer's snapshot. The conflict check sits in the version store, so it fires at write time, not commit time, and the loser aborts early instead of doing doomed work.

Row and table locks still exist for writes, managed with FIFO queues, shared and exclusive modes, and upgrades. Deadlocks are detected in a wait-for graph and broken wait-die: the older requester wins and the younger transaction aborts, with a five second timeout as the backstop. Rollback walks the write set in reverse and, per chapter 02, logs one compensation record per undone write before the ABORT record appends.

3.4 Commit, in order

Commit is an ordering exercise. The transaction appends its COMMIT record, forces the WAL through that record's LSN with an `fsync`, and only then stamps its commit timestamp into its versions, making them visible to later snapshots. Locks release after visibility, version chains prune once no live snapshot can reach the old entries, and the transaction object retires. Nothing about durability waits on data pages: they follow lazily through the buffer pool, which is exactly the window the chapter 01 schedules crash into, and the WAL closes it.

VERSION STORE

Version chains are an in-memory structure rebuilt fresh after a crash; durability belongs to the WAL and the heap. `VersionInfo` is 32 bytes: `created_by`, `deleted_by`, `begin_ts`, `end_ts`.

GC

Commit finalizes a transaction's versions and prunes chains no live snapshot can reach, so long chains decay as old readers retire.



FIG 3.3 Visibility of one RID across two snapshots. `T_a`, started between `v1`'s creation and its deletion, keeps reading `v1` even after `v2` commits; `T_b`, started later, reads `v2`.

3.5 Write-ahead log

Every mutation appends a log record before its page changes: a 32-byte header (LSN, transaction id, previous-LSN chain, payload size, type) followed by a typed payload, with before-images on updates and deletes so undo has something to restore. Records buffer in 64 KiB of user space; a commit forces the buffer through `flush_to_lsn` up to its own commit record and fsyncs the log device before the commit returns. The reader side tolerates a torn tail by construction: a record that fails its bounds check truncates the log at the last whole record, which is exactly the behavior the `torn_wal_tail` schedule sweeps.

3.6 Recovery in three passes

Recovery is ARIES reduced to what a single-node engine needs. Analysis scans forward from the last checkpoint, rebuilding the set of transactions with no durable outcome and fixing the LSN where redo must begin. Redo repeats history for every data record, committed or not, gated per page: a page whose LSN is at or past the record's already reflects it, so replay skips. The gate is what makes redo idempotent, and recovery safe to crash and re-run. Undo then walks the losers' records newest-first and applies state-checked inverses, flushing compensated pages before the closing ABORT records append, the same compensation-before-outcome ordering the live abort path follows.

Checkpoints bound the work: flush the WAL, flush dirty pages, append a checkpoint record naming the active transactions, and advance the redo anchor. The engine takes them at startup and after `DROP TABLE` rather than on a timer, a real bound on recovery time that chapter 07 lists with the other honest edges. Catalog durability rides a separate manifest file, replaced atomically: write to a temp file, fsync it, rename into place, fsync the directory.

PAGE-LSN DISCIPLINE

Redo stamps a page's LSN when it applies a record, and the forward path stamps compensation LSNs at abort time. The pairing keeps the gate honest whether a page survived the crash or was rebuilt from zero.

RECORD TYPES

BEGIN, COMMIT, ABORT; INSERT, DELETE, UPDATE with before-images; CHECKPOINT and END_CHECKPOINT. LSNs are 64-bit and start at 1; 0 is the invalid sentinel a fresh page carries.

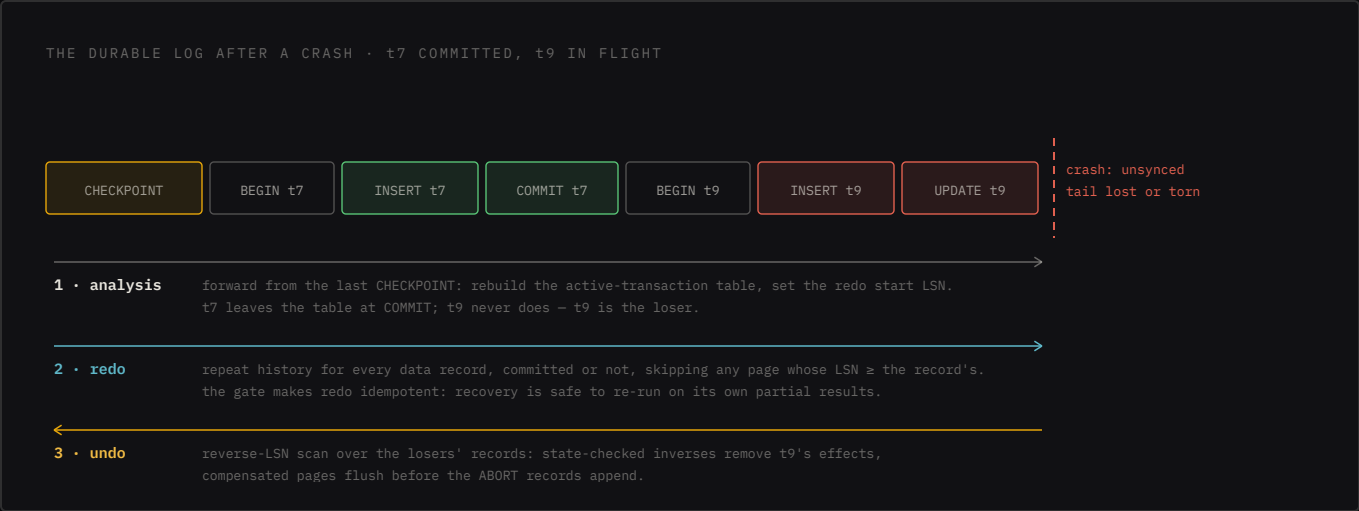


FIG 3.4 Three passes over one log. Analysis finds t9 with no outcome record; redo replays both transactions' writes; undo erases t9's, in reverse, with compensation ordering preserved.

From SQL text to a costed plan

The query layer is deliberately conventional: a hand-written recursive-descent parser, a binder that fails bad types before execution, a Selinger-style cost model with explicit constants, and volcano-model executors. No parser generator, no expression JIT, nothing on the path that is not first-party code.

4.1 Parse, then bind

The parser covers SELECT with joins (inner, left, right, cross), WHERE, GROUP BY with the standard aggregates, ORDER BY, and LIMIT/OFFSET, plus INSERT, UPDATE, DELETE, CREATE/DROP TABLE, and EXPLAIN, descending by precedence from OR down to unary operators. The binder then resolves every column against the catalog and type-checks literals against the schema, so an ill-typed INSERT or a misspelled column dies at bind time with a named error, not at execution depth.

4.2 Costing with the constants on the table

The cost model prices plans in explicit units rather than folklore. A sequential scan costs its page count plus a per-tuple CPU term; an index scan pays a logarithmic descent plus one random page fetch per match, at four times the sequential page price, which is what makes it lose on low-selectivity predicates. Selectivity defaults follow Selinger: one tenth for equality, one third for a range, refined by row counts and min/max when statistics exist. Joins are chosen the same way: an equi-join takes the hash path when build plus probe undercuts the nested-loop product, and keeps nested loop otherwise, since it works for any predicate.

CONSTANT	VALUE	MEANING
SEQ_PAGE_COST	1.0	sequential page read (the unit)
RANDOM_PAGE_COST	4.0	random page read (index fetch)
TUPLE_CPU_COST	0.01	examine one tuple
INDEX_TUPLE_COST	0.005	touch one index entry
HASH_QUAL_COST	0.025	probe one hash bucket
HASH_BUILD_COST	0.05	insert one build row
SORT_COST	0.02	per comparison, $n \cdot \log n$

4.3 Volcano execution

Plans run as a tree of iterators with `init()` and `next()`: scans (sequential and index), filter, projection, nested-loop and hash join, hash aggregation, sort, limit, and the three DML executors. One tuple flows at a time, so memory stays bounded by the pipeline's blocking operators (sort and the hash build), and every operator composes with every other.

EXPLAIN, VERBATIM FROM THE SHELL

```
entropy> EXPLAIN SELECT name
...> FROM users
...> WHERE age > 35
...> ORDER BY age DESC
...> LIMIT 10;
```

```
+-----+
| QUERY PLAN                                |
+-----+
| Query Plan:                               |
| -> Sequential Scan on users               |
|   SeqScan Cost: 1.020000                  |
|   Filter: (predicate)                     |
| -> Sort                                   |
|   Key: age DESC                           |
| -> Limit: 10                              |
| Estimated Rows: 0                         |
+-----+
```

READING THE COST

Captured after two INSERTs: 1.02 = one page + 2 rows x 0.01 examined. Small numbers, but the same arithmetic ranks plans at any scale.

HASH AGGREGATION

GROUP BY accumulates FNV-1a-hashed group keys in one pass, with SUM(INTEGER) promoted to 64-bit to dodge overflow, and AVG divided at output time.

PART III

Instruments and evidence

05 An instrument panel for crashes

06 Measured, on two machines

07 What this engine does not do

An instrument panel for crashes

A sweep of two hundred seeds prints two hundred JSONL lines, which is the right format for CI and the wrong one for a person. `entropy-tui` is the terminal front end that turns the simulator's output into something an operator can watch: a crash-recovery dashboard and a SQL console, drawn with FTXUI over the same stable JSONL schema the CLI emits.

5.1 Reproducible pixels

The interface animates in a terminal, and a PDF cannot; the frames on these pages are single stills of a moving surface. They are also reproducible artifacts in their own right: the binary's `--capture-frame` mode renders one named screen at a fixed size from the bundled demo dataset and writes the ANSI to stdout, so every image here can be regenerated byte for byte from the repository. CI builds the TUI on every push alongside the engine, so the panel cannot silently rot against the code it visualizes.

THREE SCREENS

Boot menu, crash-simulator dashboard, query console. The boot mark is a braille dot-matrix galaxy, animated in the live UI.

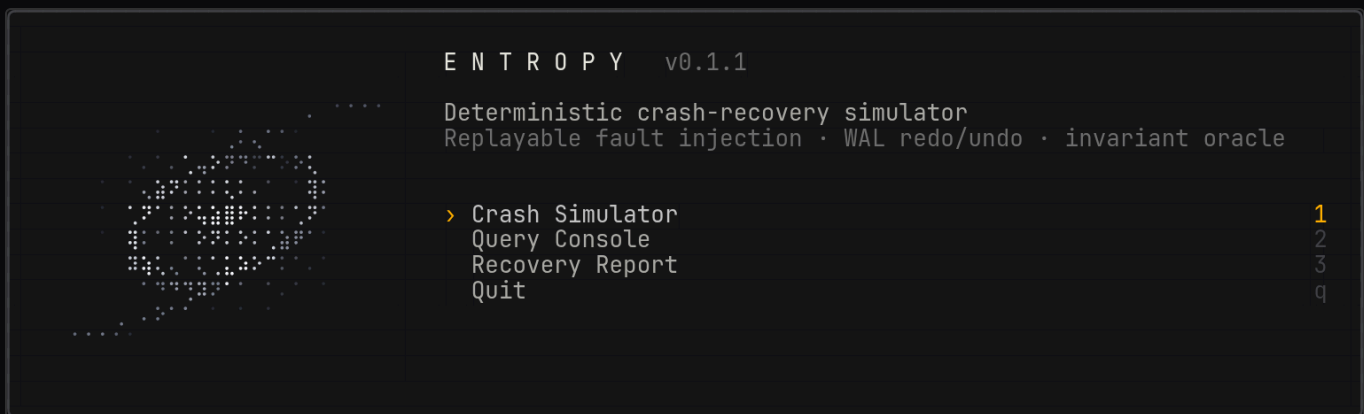


FIG 5.1 The boot screen at rest. In the terminal the galaxy rotates; the menu keys drop into the dashboard, the console, and a recovery report.

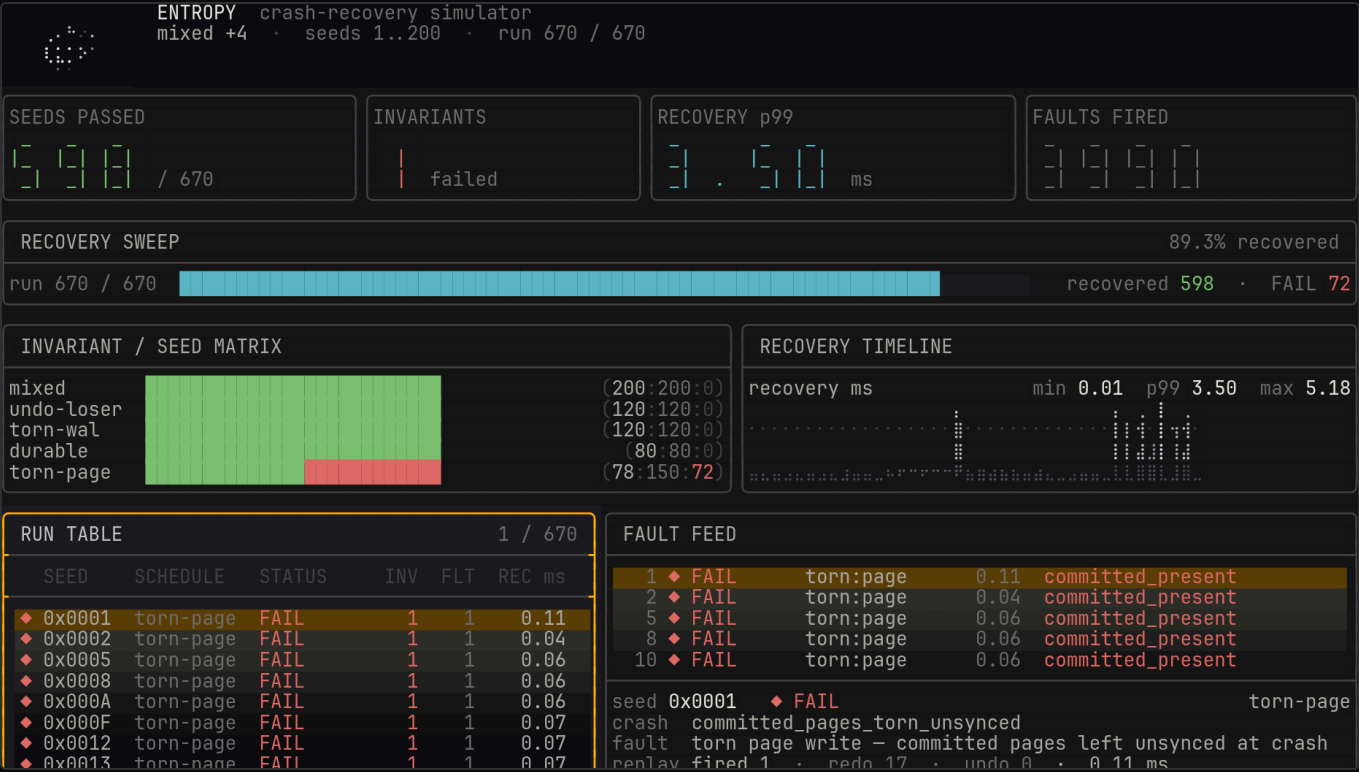


FIG 5.2 The dashboard mid-sweep. Every widget is a projection of the JSONL fields: the seed matrix buckets outcomes per schedule, the timeline plots per-run recovery_ms with p99 called out, and the fault feed streams each failing seed's invariant verdicts. The red band is deliberate: the bundled demo dataset replays the pre-fix torn-page sweep from chapter 02, 72 seeds failing committed_present.

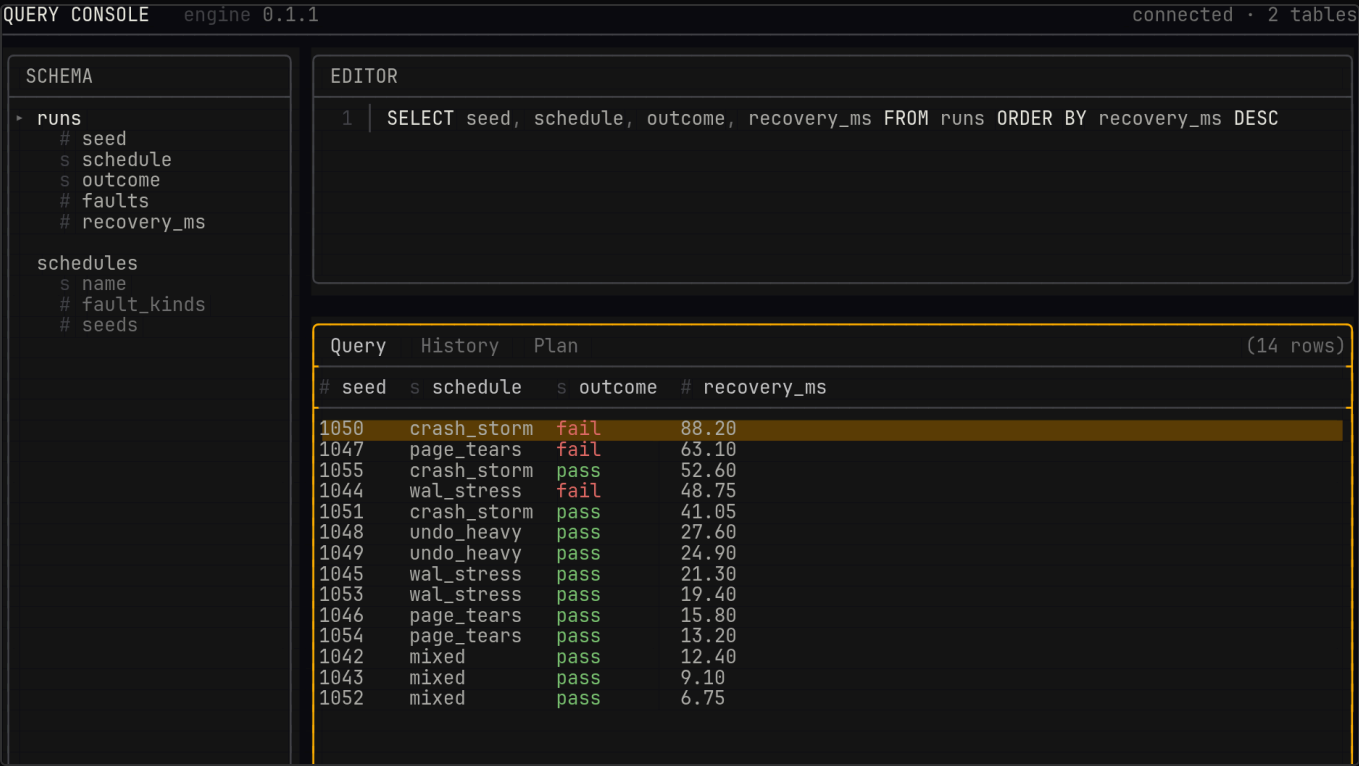


FIG 5.3 The query console is not a mock: it embeds a live entropy::Database, loads the sweep's runs and schedules as tables, and every query on this screen runs through the engine's own parser, planner, and executors. The database answers questions about its own crash record.

Measured, on two machines

Two kinds of speed matter to this project, and they are not the same. The first is how fast the simulator can interrogate the engine: on a mobile i7-1165G7, a 200-seed mixed sweep completes in 0.398 seconds of wall clock, about 500 full crash-recover-verify cycles per second, each cycle pushing roughly 1,230 logical operations through the real transaction stack before crashing it. At that rate, seed sweeps run as a CI gate instead of a nightly job.

The second is engine throughput against a production yardstick. SQLite is the honest choice of opponent for a single-node embedded engine, and the repository ships the comparison harness: identical workloads through `entropy::Database::execute` and `sqlite3_exec`, Google Benchmark timing both. Inserts run in non-durable mode on both sides (Entropy without `fsync` on commit, SQLite with `synchronous=OFF` and `journal_mode=MEMORY`), so the comparison measures engine work, not disk hardware. Point selects filter on a non-indexed column, so both engines scan.

METHOD

Batch = one transaction of N single-row INSERTs; the per-op figure is the whole batch. Point select = one SELECT by equality on a non-indexed INTEGER column, re-run per iteration.

ARTIFACTS

Run files and a summary CSV live in `docs/benchmarks/`; scripts regenerate them with one command on any machine with SQLite3 headers.

6.1 The committed run: Apple M2

The repository's benchmark record (run file `bench-20251226-214051.json`, Apple clang 17, Release -O3) puts Entropy **ahead of SQLite at 0.90x on the 1k-row transactional insert batch** and behind at 1.39x on the 10k batch. Point selects trail at 2.00x and 2.56x. For a young engine whose write path pays full WAL, MVCC versioning, and row-lock costs on every statement, holding a win on small transactional batches against SQLite is the result worth reporting, and the scan gap is the roadmap.

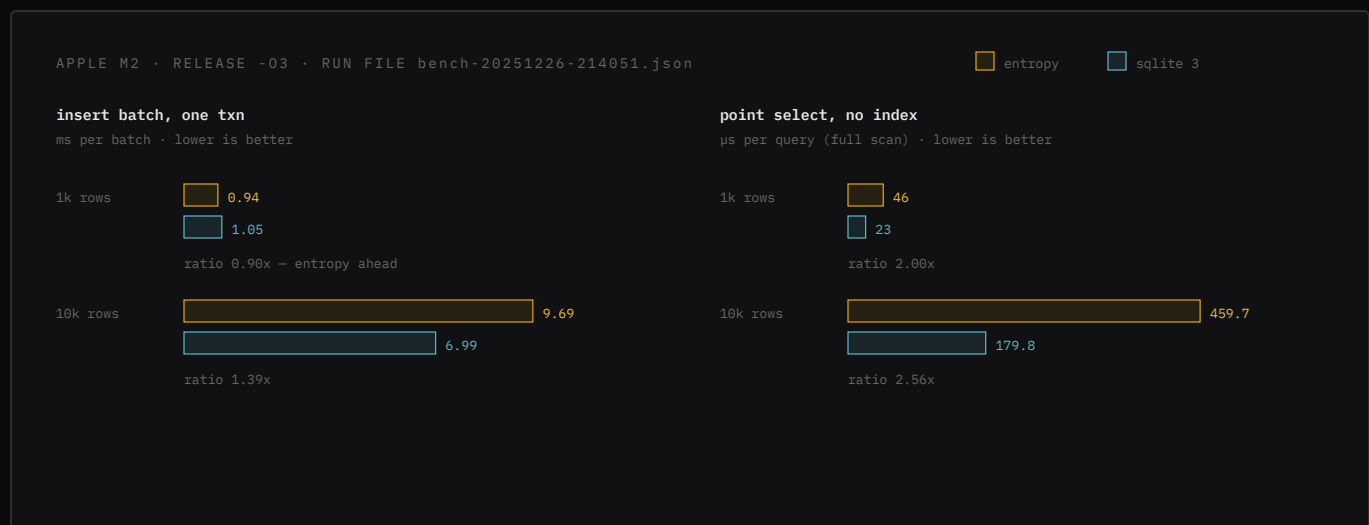


FIG 6.1 Per-op time, Entropy vs SQLite 3, from the committed M2 run. Insert panels are milliseconds per whole batch; select panels are microseconds per query. Ratios above 1.0 mean Entropy is slower.

6.2 A second machine, for honesty

The same binary rebuilt on a Debian 13 laptop (i7-1165G7, g++ 14.2.0, Release; SQLite 3.46.1) reproduces the shape and widens the gaps: mean of three repetitions puts the 1k insert batch at 1.49x, the 10k batch at 2.34x, and point selects at 3.47x and 4.44x. The conditions were deliberately unfavorable and are quoted with the numbers: CPU frequency scaling enabled and a background load average near 6.6 at capture, versus the quiet M2 run. The spread between the two machines is itself the finding; these are microbenchmarks of an engine without a benchmarking lab, and the committed run file is the number the repository stands behind.

CASE	ENTROPY	SQLITE 3	RATIO
insert batch 1k (txn)	3.58 ms	2.40 ms	1.49x
insert batch 10k (txn)	47.10 ms	20.12 ms	2.34x
point select 1k	151.5 μs	43.7 μs	3.47x
point select 10k	1.516 ms	341.6 μs	4.44x

6.3 Where the time goes

Every Entropy query takes the whole path every time: lex, parse, bind, plan, then a freshly built volcano executor tree, with an MVCC snapshot and row locks underneath. There is no prepared-statement cache and no plan reuse. SQLite re-parses on each `sqlite3_exec` call too, but its parse-to-VM pipeline has had two decades of tuning. The 10k insert batch tells the same story from the write side: Entropy's per-row cost includes a WAL record, a version-chain entry, and a lock-table touch, and those costs compound linearly where SQLite's B-tree insert loop stays flat.

The honest summary: on transactional insert batches at the 1k scale, Entropy is competitive with SQLite on one machine and within 1.5x on the other. On unindexed scans it pays 2x to 4.4x for a young executor. Nothing here is tuned yet; the cost model constants in chapter 04 are first principles, not profiles.

LINUX CAPTURE

```
g++ (Debian 14.2.0-19), --
benchmark_repetitions=3, mean real_time
quoted. cpu_scaling_enabled=true and
load_avg 6.6 in the Google Benchmark
context header: treat as directional,
not laboratory.
```

SIMULATOR THROUGHPUT

```
time entropy-sim --schedule mixed --
seeds 200: 0.39 s user, 0.398 s total,
99% cpu. All nine schedules at 200 seeds
pass on this build: 1,800 runs, 0
failures.
```

What this engine does not do

Every claim in this document was checked against the source before it was printed, and the same standard applies in reverse: these are the places the engine falls short today, each verified in code and most tracked as open issues.

7.1 Secondary indexes desynchronize on writes

The DML executors touch only the table heap: `INSERT`, `UPDATE`, and `DELETE` never update secondary indexes, so an index built before a write is stale after it (issue #11). The B+ tree also rejects duplicate keys, so building an index over a non-unique column silently drops rows and an equality lookup returns at most one match (issue #8). The heap is authoritative and sequential scans, the transactional engine, and everything the simulator tests are unaffected, but index-accelerated reads are only trustworthy on unique keys over data that has not changed since the build.

7.2 Index DDL parses but does not execute

The parser accepts `CREATE INDEX` and `DROP INDEX`, and the catalog supports both operations through the C++ API, but `Database::execute` rejects the statement types (issue #35). The same issue records that `DECIMAL`, `TIMESTAMP`, `DATE`, and `CHAR` columns cannot be declared from SQL text, and that the lexer accepts unterminated string literals and block comments instead of raising errors. Aggregates, by contrast, do run from SQL: `SUM`, `AVG`, `COUNT`, `MIN`, `MAX`, and `GROUP BY` execute through the hash aggregation path.

7.3 Durability edges

Checkpoints happen at startup and after `DROP TABLE`, not on a timer, so a long-running instance accumulates unbounded redo work between checkpoints. Data-page `sync()` in the file-backed disk manager flushes the stream without an `fsync`; durability rests on the WAL, which does `fsync`, and on redo, which rebuilds pages from it. The page free list is in-memory only, so pages freed before a restart are leaked until a durable free map exists. Version chains are also memory-resident: a restart recovers committed data exactly, but as single versions, not history.

7.4 Scope

Single-node, embedded, local-only by design: no replication, no network protocol, no subqueries, CTEs, or window functions. The simulator models faults at the storage seam (lost, torn, and erroring writes) and does not inject in-page bit flips; the checksums of chapter 02 cover that class on the read path instead.

VERIFIED LIVE, THIS BUILD

```
entropy> CREATE INDEX idx
...> ON t (id);
Error: NotSupported:
  Unsupported statement type
```

WHY PRINT THIS

A report that overstates one capability forfeits the rest. The engine's claims stand on the crash record of chapters 01 and 02; the gaps above are the work that remains, not footnotes to hide.

Reproduction

Everything in this document regenerates from the repository. Dependencies (GoogleTest, Google Benchmark, spdlog) fetch on first configure; the SQLite comparison needs system SQLite3 headers.

```
BUILD AND TEST

cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release
ctest --test-dir build --output-on-failure -C Release
```

```
CRASH SIMULATION · CHAPTERS 01-02

entropy-sim --list                # nine schedules
entropy-sim --seeds 200 --schedule mixed    # one JSONL line per run
entropy-sim --schedule live_abort_repro --seeds 40    # bug lock, ch 02 case I
entropy-sim --schedule torn_page_write --seeds 40    # bug lock, ch 02 case II
```

```
BENCHMARKS · CHAPTER 06

./scripts/bench/run.sh    # writes docs/benchmarks/runs/<timestamp>.json
# SQLite comparison is ON by default; needs sqlite3 headers
```

```
TERMINAL UI · CHAPTER 05

cmake -S . -B build -DENTROPY_BUILD_TUI=ON
entropy-tui                # live interface
entropy-tui --capture-frame dashboard --size 122x66    # the stills used here
```

```
THIS DOCUMENT

cd docs/report && npm install && npm run pdf
# -> docs/report/dist/entropy-report.pdf
```

RECORD

Bugs: issues #75, #81, #86. Fixes: commits eebf21c (PR #89) and 8bd6db2 (PR #94); checksums default-on in 62800ef (PR #96). Schedule sources: src/sim/schedule.cpp.

COLOPHON

Set in Instrument Serif, Schibsted Grotesk, and IBM Plex Mono. Composed as fixed pages in React, printed to PDF through headless Chromium. Every diagram is hand-authored SVG; the terminal stills are deterministic ANSI captures. Source: docs/report/.

LICENSE

Engine and document: MIT. SQLite is public domain; comparisons use the system library.

